Name:Mohammed Ashhad Khan

Roll no:48

Branch & Batch:AI&DS & T12

Experiment No. 1

Version Control Operations Using Git

Aim

To learn the basic Git commands used for version control, create a local Git repository, manage files using Git, and synchronize the project with GitHub.

Introduction

Git is a distributed version control system used to track changes in source code during software development. It helps developers maintain project history, collaborate with others, and restore previous versions whenever required.

GitHub is a cloud-based hosting platform that stores Git repositories online, making collaboration and backup easier.

---

1. Configure Git Username

Command

git config --global user.name "Mohammed Ashhad Khan"

Purpose

This command sets the global username for Git. Every commit made from this computer will be associated with this username.

---

2. Configure Git Email

Command

git config --global user.email "kmohammedashhad@gmail.com"

Purpose

This command sets the global email address used by Git to identify the author of each commit.

---

3. Verify Git Configuration

Command

git config --list

Purpose

Displays all Git configuration settings and verifies that the username and email are correctly configured.

---

4. Initialize Git Repository

Command

git init

Purpose

Initializes a new Git repository in the current project folder by creating a hidden **.git** directory.

---

5. Check Repository Status

Command

git status

Purpose

Displays the current status of the repository, including tracked files, modified files, and files waiting to be committed.

---

6. Stage Project Files

Command

git add .

Purpose

Adds all project files to the staging area so they are ready to be committed.

---

7. Commit Files

Command

git commit -m "Initial Commit"

Purpose

Creates the first snapshot of the project and stores it permanently in Git history.

---

8. View Commit History

Command

git log --oneline

Purpose

Displays the commit history in a short and simple format.

---

9. Add Remote GitHub Repository

Command

git remote add origin https://github.com/kmohammedashhad-design/hello.git

Purpose

Connects the local Git repository to the remote GitHub repository.

---

10. Verify Remote Repository

Command

git remote -v

Purpose

Displays the fetch and push URLs of the connected GitHub repository.

---

11. Rename Default Branch

Command

git branch -M main

Purpose

Renames the current branch to **main**.

---

12. Push Project to GitHub

Command

git push -u origin main

Purpose

Uploads the local repository to GitHub and establishes tracking between the local and remote **main** branches.

---

13. Check Repository Status After Push

Command

git status

Purpose

Verifies that the working directory is clean and that the local repository is synchronized with GitHub.

---

14. Create a New Branch

Command

git branch feature

Purpose

Creates a new development branch named **feature** without affecting the main branch.

---

15. View All Branches

Command

git branch

Purpose

Lists all available branches in the repository and indicates the currently active branch.

---

16. Switch to Feature Branch

Command

git switch feature

Purpose

Changes the current working branch from **main** to **feature**.

---

17. Merge Feature Branch

Command

git merge feature

Purpose

Merges all changes from the feature branch into the current branch.

---

18. Delete Feature Branch

Command

git branch -d feature

Purpose

Deletes the feature branch after it has been successfully merged.

---

19. Pull Latest Changes

Command

git pull

Purpose

Downloads the latest changes from the GitHub repository and updates the local project.

---

20. View File Differences

Command

git diff

Purpose

Displays the differences between the current working directory and the most recent committed version.

---

21. Restore a File

Command

git restore debug.py

Purpose

Restores the file to its previous committed version by discarding uncommitted changes.

---

22. Unstage a File

Command

git restore --staged debug.py

Purpose

Removes the file from the staging area without deleting the changes made to it.



```
https://learn.microsoft.com/windows/terminal/troubleshooting#black-lines-in-powershell-51-6x-70
Install the latest version by running: 'Install-Module PSReadLine -MinimumVersion 2.0.3 -Scope CurrentUser -Force'
After the installation, open a new terminal tab.

(base) (.venv) PS C:\code\hello> git init
Initialized empty Git repository in C:/code/hello/.git/
(base) (.venv) PS C:\code\hello> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/
        __pycache__/
        app.py
        codes.py
        debug.py
        for.py
        image.webp
        menuif.py
        mohit.py
        rester _upd.py
        simplex.py
        simplex2.py
        url shortner.py

nothing added to commit but untracked files present (use "git add" to track)
```

```
        __pycache__/
        app.py
        codes.py
        debug.py
        for.py
        image.webp
        menuif.py
        mohit.py
        rester _upd.py
        simplex.py
        simplex2.py
        url shortner.py

nothing added to commit but untracked files present (use "git add" to track)
(base) (.venv) PS C:\code\hello> git add .
warning: in the working copy of '.idea/inspectionProfiles/profiles_settings.xml', LF will be replaced by CRLF the next time Git touches it
(base) (.venv) PS C:\code\hello> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .idea/.gitignore
        new file:   .idea/hello.iml
        new file:   .idea/inspectionProfiles/profiles_settings.xml
        new file:   .idea/misc.xml
        new file:   .idea/modules.xml
        new file:   .idea/vcs.xml
        new file:   __pycache__/app.cpython-313.pyc
        new file:   app.py
        new file:   codes.py
        new file:   debug.py
```

```
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .idea/.gitignore
        new file:   .idea/hello.iml
        new file:   .idea/inspectionProfiles/profiles_settings.xml
        new file:   .idea/misc.xml
        new file:   .idea/modules.xml
        new file:   .idea/vcs.xml
        new file:   __pycache__/app.cpython-313.pyc
        new file:   app.py
        new file:   codes.py
        new file:   debug.py
        new file:   for.py
        new file:   image.webp
        new file:   menuif.py
        new file:   mohit.py
        new file:   rester _upd.py
        new file:   simplex.py
        new file:   simplex2.py
        new file:   url shortner.py

(base) (.venv) PS C:\code\hello> git commit -m "Initial Commit"
[main (root-commit) a124eb3] Initial Commit
 18 files changed, 929 insertions(+)
 create mode 100644 .idea/.gitignore
 create mode 100644 .idea/hello.iml
 create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
 create mode 100644 .idea/misc.xml
 create mode 100644 .idea/modules.xml
 create mode 100644 .idea/vcs.xml
```

```
(base) (.venv) PS C:\code\hello> git commit -m "Initial Commit"
[main (root-commit) a124eb3] Initial Commit
 18 files changed, 929 insertions(+)
 create mode 100644 .idea/.gitignore
 create mode 100644 .idea/hello.iml
 create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
 create mode 100644 .idea/misc.xml
 create mode 100644 .idea/modules.xml
 create mode 100644 .idea/vcs.xml
 create mode 100644 __pycache__/app.cpython-313.pyc
 create mode 100644 app.py
 create mode 100644 codes.py
 create mode 100644 debug.py
 create mode 100644 for.py
 create mode 100644 image.webp
 create mode 100644 menuif.py
 create mode 100644 mohit.py
 create mode 100644 rester _upd.py
 create mode 100644 simplex.py
 create mode 100644 simplex2.py
 create mode 100644 url shortner.py
(base) (.venv) PS C:\code\hello> git log --oneline
a124eb3 (HEAD -> main) Initial Commit
(base) (.venv) PS C:\code\hello> git remote add origin https://github.com/yourusername/hello.git
(base) (.venv) PS C:\code\hello> git remote -v
origin  https://github.com/yourusername/hello.git (fetch)
origin  https://github.com/yourusername/hello.git (push)
(base) (.venv) PS C:\code\hello> git branch -M main
(base) (.venv) PS C:\code\hello> git push -u origin main
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/yourusername/hello.git/' not found
```

hello > debug.py

Project rester _upd.py debug.py codes.py for.py url shortner.py

Terminal Local

```
(base) (.venv) PS C:\code\hello> git status
On branch main
nothing to commit, working tree clean
(base) (.venv) PS C:\code\hello> git add .
(base) (.venv) PS C:\code\hello> git status
On branch main
nothing to commit, working tree clean
(base) (.venv) PS C:\code\hello> git commit -m "Initial Commit"
On branch main
nothing to commit, working tree clean
(base) (.venv) PS C:\code\hello> git remote remove origin
(base) (.venv) PS C:\code\hello> git remote add origin https://github.com/kmohammedashhad-design/hello.git
(base) (.venv) PS C:\code\hello> git remote -v
origin  https://github.com/kmohammedashhad-design/hello.git (fetch)
origin  https://github.com/kmohammedashhad-design/hello.git (push)
(base) (.venv) PS C:\code\hello> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 23, done.
Counting objects: 100% (23/23), done.
Delta compression using up to 4 threads
Compressing objects: 100% (20/20), done.
Writing objects: 100% (23/23), 27.08 KiB | 1.50 MiB/s, done.
Total 23 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kmohammedashhad-design/hello.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(base) (.venv) PS C:\code\hello> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
(base) (.venv) PS C:\code\hello>
```

Conclusion

This experiment successfully demonstrated the fundamental operations of Git and GitHub. The Git environment was configured using a username and email, a local repository was initialized, project files were staged and committed, and the repository was successfully connected to GitHub. The experiment also covered essential Git operations such as checking repository status, viewing commit history, creating and switching branches, merging branches, restoring files, and synchronizing the local repository with the remote GitHub repository. Through this experiment, a clear understanding of version control and collaborative source code management was achieved.